NOTE WebSockets are a protocol that sit on top of HTTP, allowing for asynchronous, bidirectional communication on the web. They require JavaScript to work in the browser.

B.4 Readying Django for production

Once your infrastructure is in place, you have to deploy your project for the first time. Your code needs to be put on the server, your server may need configuration, your production database needs to be synchronized with your ORM through the `migrate` management command, your static files need to be copied from your project folder to the web server's folder, and possibly more. This section walks you through each of these tasks.

B.4.1 Environment-specific configuration

At minimum, you have two environments for your code: development (usually your local machine) and production (available to the world). More complex setups are possible, and quite common, in industry. In large organizations, you might have a local machine, a development server, multiple levels of testing servers, and then production.

Some configuration in settings.py is common to all deployment environments, but some is environment specific. For example, `DEBUG` should be `True` in dev and `False` in production. There are two general approaches to dealing with these differences: using environment variables and importing values.

There is a reason settings.py ends in *.py*: it is a Python file. Throughout the book, you have added Python variables as configuration values in settings.py, but you can also run code. You don't want to do this very much, but a few lines can offer you conditional configuration based on your deployment target.

Your operating system allows for the setting of environment variables that are accessible within a program. If you are on a Unix-based system and running bash (and most other shells), the `env` command shows how many variables are currently set. In Python, the `environ` dictionary inside the `os` module contains all of these environment variables. Inside settings.py, you can set a value based on the contents of `os.environ`. This is particularly handy for things like passwords and keys that you don't want committed to your code repo. Here is an example that sets the `SECRET_KEY` value:

```
# RiffMates/RiffMates/settings.py
...
import os
...
SECRET_KEY = os.environ["RIFFMATES_SECRET"]
```

Instead of using environment variables, you can manage the settings.py file itself. A low-tech solution is to have settings_dev.py and settings_prod.py and, as part of your deployment, copying the appropriate file to settings.py. There are a couple of drawbacks to this approach: (1) many values will be common to both, and (2) you still have the problem of not storing secrets in your repo, which means you may not want to store either of these files in your code repository.